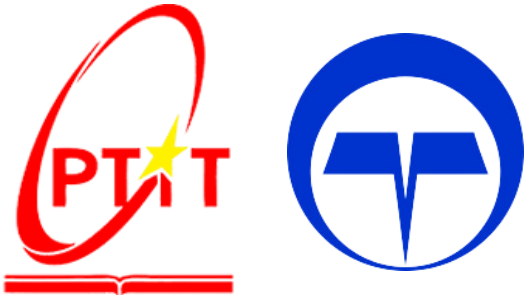
HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
VIỆN KHOA HỌC KỸ THUẬT BƯU ĐIỆN



MÔN: PHÁT TRIỂN PHẦN MỀM HƯỚNG DỊCH VỤ

BÀI TẬP LỚN: HỆ THỐNG WATTPRINT

Theo dõi điện năng từng thiết bị và trợ lý năng lượng cá nhân hóa cho hộ gia đình

Giảng viên	**:**	**Nguyễn Việt Dũng**
Mã lớp	**:**	**INT1448-20261-01**
Nhóm	**:**	**Nhóm 6**
Họ và tên	**:**	**MSV**
Nguyễn Trường Giang	:	B23DCCC055
Lê Thị Hà	:	B23DCCC056
Nguyễn Thị Quỳnh Trang	:	B23DCCC160

HÀ NỘI, 2026

TÓM TẮT

Báo cáo phân tích nghiệp vụ và thiết kế kiến trúc microservices cho WattPrint, hệ thống giúp hộ gia đình xem điện năng từng thiết bị chỉ từ một điểm đo ở aptomat tổng và nhận gợi ý tiết kiệm từ trợ lý Energy Copilot. Từ chín quy tắc nghiệp vụ, nhóm tách hệ thống thành bốn service là core, nilm, insight và copilot, dựa trên data ownership, workload và fault isolation.

Các service đứng sau một API Gateway, mỗi service có database riêng, gọi nhau đồng bộ theo một chiều và truyền event bất đồng bộ qua RabbitMQ với Transactional Outbox/Inbox. Báo cáo ghi lại các phương án đã cân nhắc, lý do chọn và các đánh đổi nhóm chấp nhận. Thiết kế API, kiểm thử độ tin cậy có số liệu, triển khai và đánh giá sẽ trình bày ở các chương sau.

Từ khóa: microservices, SOA, API Gateway, NILM, RabbitMQ, transactional outbox, Energy Copilot.

MỤC LỤC

TÓM TẮT i
DANH MỤC TỪ VIẾT TẮT VÀ THUẬT NGỮ v
CHƯƠNG 1. GIỚI THIỆU 1
1.1. Bối cảnh 1
1.2. Giải pháp WattPrint 1
1.3. Phạm vi của bài tập lớn 2
1.4. Yêu cầu của đề bài 2
1.5. Cấu trúc báo cáo 2
CHƯƠNG 2. PHÂN TÍCH NGHIỆP VỤ 4
2.1. Người dùng và các bên liên quan 4
2.2. Các nhóm nghiệp vụ 4
2.3. Quy tắc nghiệp vụ 5
2.4. Yêu cầu phi chức năng 6
2.5. Nguồn dữ liệu sensor và dữ liệu thử 7
CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ KIẾN TRÚC 8
3.1. Tổng quan kiến trúc 8
3.2. Bối cảnh hệ thống 8
3.3. Tách hệ thống thành các service 9
3.4. Các khối triển khai 11
3.5. Giao tiếp giữa các service 12
3.6. Luồng nghiệp vụ nhiều bước và xử lý lỗi giữa chừng 14
3.7. Dữ liệu 16
3.8. Gateway và bảo mật 17
3.9. Công nghệ sử dụng 18
3.10. Thành phần bên trong từng service 19
CHƯƠNG 4. ĐÁNH ĐỔI VÀ RỦI RO CỦA KIẾN TRÚC 24
4.1. Các quyết định chính và đánh đổi 24
4.2. Rủi ro còn lại và hướng xử lý 25
TÀI LIỆU THAM KHẢO 26

DANH MỤC BẢNG

Bảng 2.1. Người dùng, nguồn dữ liệu và hệ thống bên ngoài 4
Bảng 2.2. Sáu nhóm nghiệp vụ của WattPrint 4
Bảng 2.3. Quy tắc nghiệp vụ 5
Bảng 2.4. Yêu cầu phi chức năng và ảnh hưởng đến kiến trúc 6
Bảng 3.1. Các phương án tách service 10
Bảng 3.2. Service, trách nhiệm, dữ liệu riêng và đối tượng gọi tới 10
Bảng 3.3. Các sự kiện trên exchange `wattprint.events` (kiểu topic) 12
Bảng 3.4. Công nghệ sử dụng và lý do chọn 18
Bảng 4.1. Quyết định kiến trúc và đánh đổi 24
Bảng 4.2. Rủi ro và hướng xử lý 25

DANH MỤC HÌNH

Hình 3.1. Kiến trúc tổng quan của WattPrint. Mũi tên liền là call đồng bộ, mũi tên đứt là sự kiện bất đồng bộ qua RabbitMQ 8

Hình 3.2. Sơ đồ System Context của WattPrint..... 9

Hình 3.3. Sơ đồ Container của WattPrint. Nét đứt là message AMQP: mũi tên đi tới queue là bên gửi, đi ra khỏi queue là bên nhận.....11

Hình 3.4. Một sự kiện đi qua RabbitMQ: outbox, exchange, queue, consumer, DLQ.14

Hình 3.5. Luồng ingest dữ liệu sensor và chạy inference NILM, có trạng thái PENDING_INFERENCE khi nilm không phản hồi 15

Hình 3.6. Luồng kích hoạt kế hoạch tiết kiệm (saga): thành công, hoàn tác theo thứ tự ngược và PENDING 16

Hình 3.7. Sơ đồ Component của service core.....20

Hình 3.8. Sơ đồ Component của service nilm 21

Hình 3.9. Sơ đồ Component của service insight.....22

Hình 3.10. Sơ đồ Component của service copilot.....23

DANH MỤC TỪ VIẾT TẮT VÀ THUẬT NGỮ

Từ	Nghĩa trong báo cáo
API	Giao diện lập trình ứng dụng (Application Programming Interface)
AMQP	Giao thức truyền message mà RabbitMQ sử dụng (Advanced Message Queuing Protocol)
BR	Quy tắc nghiệp vụ (Business Rule), đánh mã BR1 đến BR9 ở mục 2.3
C4	Mô hình mô tả kiến trúc phần mềm theo bốn cấp: Context, Container, Component, Code
CQRS	Pattern tách luồng ghi và luồng đọc dữ liệu (Command Query Responsibility Segregation)
CT	Sensor dòng điện kẹp ngoài dây dẫn (Current Transformer)
DLQ	Hàng đợi lưu các message xử lý lỗi quá số lần retry (Dead Letter Queue)
EVN	Tập đoàn Điện lực Việt Nam
Gateway	API Gateway, cổng giao tiếp duy nhất đón request từ bên ngoài vào hệ thống
JWT	Token xác thực có chữ ký số do Keycloak cấp (JSON Web Token)
JWKS	Tập khóa công khai dùng để xác thực chữ ký JWT (JSON Web Key Set)
LLM	Mô hình ngôn ngữ lớn (Large Language Model)
M&V	Đo lường và kiểm chứng mức tiết kiệm điện (Measurement and Verification)
NILM	Kỹ thuật tách điện năng từng thiết bị từ công suất tổng (Non-Intrusive Load Monitoring)
OIDC	Giao thức xác thực OpenID Connect
Outbox, Inbox	Hai bảng trong database giúp gửi event không mất (outbox) và nhận event không trùng (inbox), xem mục 3.5
PII	Thông tin định danh cá nhân (Personally Identifiable Information)
RAG	Kỹ thuật truy xuất tri thức trước khi sinh câu trả lời (Retrieval-Augmented Generation)
REFIT	Bộ dữ liệu công khai đo điện năng hộ gia đình tại Vương quốc Anh
REST	Kiến trúc thiết kế API qua HTTP (Representational State Transfer)
SOA	Kiến trúc hướng dịch vụ (Service-Oriented Architecture)

CHƯƠNG 1. GIỚI THIỆU

1.1. Bối cảnh

Tiền điện là khoản chi cố định hằng tháng, nhưng hộ gia đình thường chỉ thấy tổng số tiền cuối tháng chứ không biết thiết bị nào tiêu thụ bao nhiêu. Một thử nghiệm trên 45.000 hộ ở Việt Nam cho thấy chỉ cần gửi ước tính tiền điện theo ngày, nhóm nhận thông tin đã giảm tiêu thụ so với nhóm đối chứng [1]. Phản hồi như vậy đủ để đổi thói quen, nhưng muốn biết nên tiết kiệm ở đâu thì cần số liệu từng thiết bị. Một nghiên cứu ở Mỹ cho thấy người dùng ước tính điện năng của thiết bị thấp hơn thực tế khoảng 2,8 lần, và sai nhiều nhất ở những thiết bị tốn điện nhất [2].

Công cụ hiện có chưa đáp ứng được nhu cầu này. Ứng dụng của EVN chỉ hiển thị tổng điện năng của cả hộ theo ngày [3]. Đo riêng từng thiết bị bằng ổ cắm thông minh thì mỗi ổ khoảng 235.000 đồng [4], và với khoảng 10 điểm đo cùng bộ điều khiển trung tâm [5] hộ phải chi từ 4 đến 10 triệu đồng, tự lắp và cấu hình. Ổ cắm cũng không đo được thiết bị đấu thẳng vào đường dây âm tường như điều hòa hay đèn trần.

Câu hỏi của dự án là làm sao ước tính điện năng từng thiết bị với chi phí phần cứng thấp hơn nhiều. Nhóm chọn Non-Intrusive Load Monitoring (NILM), do Hart đề xuất năm 1992 [6]: chỉ đo tổng công suất tại aptomat chính, rồi dùng mô hình học máy tách ra công suất từng thiết bị.

1.2. Giải pháp WattPrint

WattPrint có bốn khối chức năng, đi từ dữ liệu sensor thô đến khuyến nghị cụ thể:

1. Module NILM dùng NILMFormer [7], mô hình Transformer tách tải tốt cả khi tổng công suất biến động mạnh, kèm bước calibration theo từng hộ.
2. Module dashboard và hóa đơn hiển thị điện năng từng thiết bị theo giờ, ngày, tuần, tháng, quy ra tiền theo biểu giá sinh hoạt 5 bậc.
3. Module phân tích và dự báo cảnh báo thiết bị chạy lâu bất thường, phát hiện tiêu thụ ngầm (standby), so sánh với tháng trước và với các hộ tương đồng, dự báo hóa đơn cuối tháng và thời điểm chạm bậc giá kế tiếp.
4. Energy Copilot trả lời câu hỏi của người dùng (pull) và chủ động gợi ý tiết kiệm (push). Mọi con số do các tool tính từ dữ liệu của hộ, LLM chỉ diễn giải. Copilot còn lập kế hoạch tiết kiệm và so sánh kết quả trước và sau khi áp dụng.

Về tác động kỳ vọng, các nghiên cứu trước ghi nhận phản hồi điện năng giúp hộ giảm khoảng 2-4% [8], và thử nghiệm trên khoảng 1.000 hộ ở Đà Nẵng ghi nhận mức

giảm 4-7% [9]. Đây là kết quả của các nghiên cứu đó, chưa phải kết quả đo của WattPrint.

1.3. Phạm vi của bài tập lớn

WattPrint hiện chạy dưới dạng monolith, một ứng dụng FastAPI có đủ các chức năng trên. Bài tập lớn tách monolith này thành các service độc lập, nên phần phân tích tập trung vào việc tách ở đâu và các service phối hợp ra sao. Phạm vi gồm:

- Bốn service nghiệp vụ sau một API Gateway, kèm xác thực, database riêng cho từng service và message broker. Mỗi service có Dockerfile riêng, môi trường demo dựng bằng Docker Compose.
- Service `nilm` chỉ chạy inference với trọng số NILMFormer đã nạp sẵn. Huấn luyện và đánh giá độ chính xác của mô hình nằm ngoài phạm vi.
- Nhóm không có sensor phần cứng nên dùng bộ dữ liệu công khai REFIT [10], phát lại (replay) qua gateway (mục 2.5).
- Hai ứng dụng client sẵn có (app Expo và web Next.js) giữ nguyên, chỉ đổi địa chỉ API về gateway.

1.4. Yêu cầu của đề bài

Đề bài yêu cầu hệ thống microservices có API Gateway, đóng gói bằng container. Với nhóm ba người, cần tối thiểu ba service nghiệp vụ không tính gateway, mỗi service có mã nguồn, dữ liệu riêng và ít nhất bốn REST endpoint kèm đặc tả OpenAPI. Call giữa các service phải có timeout và retry, và ít nhất một luồng đi qua hai service phải xử lý được lỗi giữa chừng. Phần kiểm thử gồm Postman collection và ít nhất hai tình huống lỗi có số liệu trước và sau khi áp dụng cơ chế fault tolerance.

Nhóm làm thêm cả bốn mục điểm cộng: event qua RabbitMQ với consumer idempotent và DLQ, xác thực JWT tại gateway, circuit breaker và bulkhead, CI chạy test tự động. Chương 3 và 4 cho thấy từng yêu cầu ảnh hưởng đến kiến trúc thế nào.

1.5. Cấu trúc báo cáo

Chương 2 phân tích nghiệp vụ: tác nhân, sáu nhóm chức năng, chín quy tắc nghiệp vụ, yêu cầu phi chức năng và nguồn dữ liệu sensor. Chương 3 trình bày kiến trúc theo mô hình C4, từ tiêu chí tách service, các container, giao tiếp đồng bộ và bất đồng bộ, luồng đa bước, dữ liệu, gateway và bảo mật, công nghệ, đến các module trong từng service. Chương 4 liệt kê các đánh đổi và rủi ro. Thiết kế chi tiết API, độ tin cậy có số

liệu, triển khai, kiểm thử và đánh giá sẽ bổ sung ở các chương sau khi hoàn tất phần hiện thực.

CHƯƠNG 2. PHÂN TÍCH NGHIỆP VỤ

2.1. Người dùng và các bên liên quan

Hệ thống có hai nhóm người dùng, một nguồn dữ liệu sensor tự động và năm hệ thống bên ngoài (Bảng 2.1).

Bảng 2.1. Người dùng, nguồn dữ liệu và hệ thống bên ngoài

Đối tượng	Vai trò	Tương tác chính
Thành viên hộ gia đình	Người dùng cuối. Chủ hộ tạo hồ sơ hộ gia đình và mời thành viên	Xem điện năng từng thiết bị, hóa đơn, cảnh báo. Hỏi đáp với Energy Copilot và nhận gợi ý qua push notification
Nhóm vận hành	Quản trị hệ thống WattPrint	Quản lý hộ gia đình, công tơ, phiên bản model. Theo dõi log, xử lý message trong DLQ và nhận cảnh báo sự cố qua Telegram
Sensor ở aptomat (REFIT replayer trong đồ án)	Nguồn cấp dữ liệu sensor	Định kỳ gửi từng batch công suất tổng lên hệ thống, xác thực bằng API key riêng của thiết bị
Nhà cung cấp LLM	Hệ thống ngoài	Sinh câu trả lời, xử lý tool calling và tạo embedding cho Copilot. Chỉ nhận prompt đã ẩn PII
Open-Meteo	Hệ thống ngoài	Cung cấp dữ liệu nhiệt độ và độ ẩm theo giờ phục vụ phân tích phụ tải và gợi ý theo thời tiết
Expo Push, Telegram	Hệ thống ngoài	Gửi push notification tới điện thoại người dùng và gửi cảnh báo cho nhóm vận hành
Google Identity	Hệ thống ngoài	Xác thực đăng nhập bằng tài khoản Google (OIDC), liên kết qua Keycloak

2.2. Các nhóm nghiệp vụ

Nghiệp vụ chia thành sáu nhóm theo đường đi của dữ liệu, từ thiết lập hộ gia đình đến khi sinh khuyến nghị tiết kiệm (Bảng 2.2). Đường đi này là cơ sở để chia service ở Chương 3.

Bảng 2.2. Sáu nhóm nghiệp vụ của WattPrint

Nhóm	Nội dung	Dữ liệu tạo ra
1. Quản lý hộ gia đình và thiết bị	Tạo hồ sơ hộ gia đình, mời thành viên bằng mã, ghép nối công tơ, khai báo danh mục thiết bị theo phòng	Hồ sơ hộ, thành viên, công tơ, thiết bị
2. Ingest dữ liệu sensor	Nhận batch công suất tổng từ sensor, kiểm tra tính liên tục, lưu chuỗi thời gian	Chuỗi dữ liệu sensor theo thời gian
3. Tách điện năng (NILM)	Chạy inference ước tính công suất từng thiết bị từ dữ liệu sensor tổng, hiệu chỉnh theo hộ, quản lý phiên bản model	Công suất ước tính theo thiết bị, lịch sử chạy inference

Nhóm	Nội dung	Dữ liệu tạo ra
4. Tính hóa đơn	Tổng hợp kWh theo giờ, ngày, tháng và tính tiền điện theo biểu giá 5 bậc	Hóa đơn, phiên bản biểu giá
5. Phân tích và dự báo	Cảnh báo bất thường, phát hiện tiêu thụ ngầm, dự báo hóa đơn và bậc giá, ngân sách, so sánh hộ tương đồng, what-if, M&V	Cảnh báo, bản ghi dự báo, baseline M&V
6. Trợ lý Copilot	Hỏi đáp tự nhiên, gợi ý chủ động, kế hoạch tiết kiệm, thông báo, ghi nhận phản hồi	Hội thoại, gợi ý, kho tri thức RAG

Luồng chính đi từ trên xuống dưới Bảng 2.2. Sensor gửi batch công suất tổng, hệ thống lưu rồi tách thành điện năng từng thiết bị và tính tiền (nhóm 2 đến 4). Kết quả tách là đầu vào để phát hiện bất thường và dự báo (nhóm 5). Cuối cùng Copilot dùng kết quả phân tích để trả lời và chủ động gợi ý (nhóm 6). Hai chỗ nối đi qua nhiều service nên có thể lỗi giữa chừng: từ ingest sang inference NILM (nhóm 2 gọi nhóm 3) và từ kế hoạch tiết kiệm sang phân tích (nhóm 6 gọi nhóm 5). Mục 3.6 thiết kế cơ chế xử lý riêng cho hai luồng này.

2.3. Quy tắc nghiệp vụ

Chín quy tắc ở Bảng 2.3. Một quy tắc chỉ được đưa vào khi kiểm chứng được bằng code: request vi phạm thì service từ chối (thường trả 4xx kèm lý do) hoặc rẽ sang nhánh xử lý khác.

Bảng 2.3. Quy tắc nghiệp vụ

Mã	Quy tắc	Lý do
BR1	Mỗi tài khoản chỉ thuộc đúng một hộ (`home`). Thành viên mới tham gia bằng mã mời chỉ dùng một lần và có thời hạn	Ngăn một tài khoản xem dữ liệu của nhiều hộ. Mã mời bị lộ chỉ có hiệu lực trong thời gian ngắn
BR2	Mỗi công tơ chỉ ghép nối với một hộ bằng mã pairing. Hệ thống chỉ ingest dữ liệu sensor từ công tơ đã ghép nối kèm API key của thiết bị	Dữ liệu sensor luôn gắn đúng hộ
BR3	Batch dữ liệu sensor phải idempotent và liên tục: gửi lặp cùng `Idempotency-Key` không ghi hai lần, khoảng thời gian của batch không được hở hoặc chồng lấp với batch liền trước	Mạng lỗi hoặc sensor retry không làm sai lệch sản lượng điện
BR4	Tiền điện tính theo biểu giá sinh hoạt 5 bậc, áp dụng đúng phiên bản biểu giá có hiệu lực trong kỳ lập hóa đơn	Hóa đơn đã chốt không đổi khi cập nhật biểu giá mới
BR5	Dự báo hóa đơn cần ít nhất 7 ngày dữ liệu, so sánh hộ tương đồng cần ít nhất 10 hộ, M&V cần ít nhất 14 ngày. Không đủ điều kiện thì trả lỗi 4xx kèm lý do	Không đưa ra con số dự báo thiếu cơ sở thống kê. Giữ ẩn danh khi so sánh giữa các hộ
BR6	Energy Copilot chỉ mở cho thuê bao gói Pro. Gói thường vẫn dùng đầy đủ dashboard và các tính năng phân tích	Mô hình kinh doanh có hai gói thuê bao

Mã	Quy tắc	Lý do
BR7	Thông báo chủ động giới hạn tối đa N thông báo mỗi ngày (N cấu hình được) và không gửi vào khung giờ nghỉ (`quiet hours`). Gợi ý bị từ chối sẽ bị hạ thứ hạng ở lần sau	Hạn chế thông báo dồn dập
BR8	Mọi con số trong câu trả lời của Copilot đều do các tool tính từ dữ liệu của hộ, LLM chỉ diễn giải. Câu trả lời sai số liệu, đơn vị hoặc thiếu nguồn trích dẫn sẽ bị loại trước khi gửi	Mọi con số tư vấn chính xác và kiểm chứng được
BR9	PII phải được ẩn trước khi gửi prompt cho nhà cung cấp LLM. Copilot chỉ được truy xuất dữ liệu thuộc hộ của người đang hỏi	Bảo vệ quyền riêng tư và cô lập dữ liệu giữa các hộ

Chín quy tắc cũng cho thấy ranh giới dữ liệu. BR1 đến BR4 gắn với dữ liệu vận hành (hộ, công tơ, dữ liệu sensor, hóa đơn) nên cần một `source of truth` duy nhất. BR5 thuộc phần phân tích. BR6 đến BR9 thuộc trợ lý AI, gồm quyền truy cập và kiểm duyệt. Chương 3 dùng chính ranh giới này để tách service.

2.4. Yêu cầu phi chức năng

Năm yêu cầu phi chức năng ở Bảng 2.4 định hình kiến trúc.

Bảng 2.4. Yêu cầu phi chức năng và ảnh hưởng đến kiến trúc

Yêu cầu	Diễn giải	Ảnh hưởng đến kiến trúc
Không mất, không trùng dữ liệu sensor	Sensor thường retry batch khi mất mạng. Sai lệch ở dữ liệu sensor thô làm sai mọi kết quả tính tiền và phân tích phía sau	Kiểm tra `Idempotency-Key`, kiểm tra batch liên tục, ghi dữ liệu và event trong cùng transaction (Transactional Outbox)
Cô lập lỗi giữa các thành phần	LLM phản hồi chậm hoặc model NILM quá tải không được làm gián đoạn luồng ingest dữ liệu sensor và dashboard	Tách service theo đặc thù lỗi. Áp dụng timeout, retry, circuit breaker và trạng thái chờ (`PENDING`)
Bảo mật và quyền riêng tư	Chuỗi công suất tiêu thụ phản ánh chi tiết giờ giấc sinh hoạt của gia đình	Xác thực JWT tại gateway. Ẩn PII trước khi gọi LLM. Mỗi service một database và một DB role riêng
Dựng được toàn bộ hệ thống trên một máy để demo	Đề bài yêu cầu chạy demo trực tiếp toàn hệ thống	Mỗi service có Dockerfile và endpoint `/health`, `/ready` riêng. Môi trường demo gom bằng `docker-compose.yml` và giới hạn tổng số container
Vừa sức vận hành của nhóm ba người	Mỗi công nghệ thêm vào đều làm tăng chi phí học, cấu hình và gỡ lỗi	Thống nhất một ngôn ngữ cho backend (Python), dùng thư viện chung, ưu tiên phần mềm có sẵn thay vì tự viết

2.5. Nguồn dữ liệu sensor và dữ liệu thử

Ngoài đời, mỗi hộ lắp một sensor dòng điện (CT) ở aptomat tổng, ví dụ Shelly EM hoặc ESP32 chạy ESPHome. Sensor đo công suất tổng và gửi từng batch qua HTTPS tới API Gateway. Nhóm không có phần cứng này nên dùng REFIT replayer.

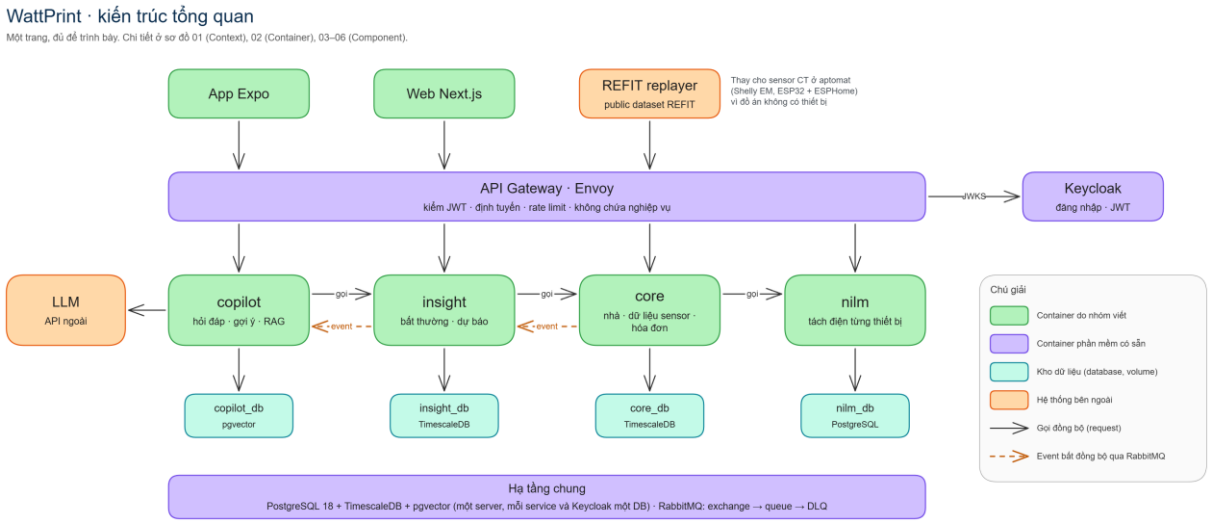
REFIT [10] là bộ dữ liệu công khai về điện năng hộ gia đình ở Vương quốc Anh, đo liên tục khoảng hai năm với chu kỳ 8 giây, gồm công suất tổng và công suất từng thiết bị. REFIT replayer là script Python đọc các file CSV của REFIT rồi gửi batch qua gateway, dùng đúng REST endpoint, API key thiết bị và `Idempotency-Key` như sensor thật, có tùy chọn tua nhanh khi demo. Nhờ vậy các service phía sau gateway xử lý hai nguồn như nhau. Trên sơ đồ, replayer được vẽ như nguồn dữ liệu bên ngoài, có ghi chú đây là công cụ phát lại từ dataset công khai.

CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ KIẾN TRÚC

3.1. Tổng quan kiến trúc

Kiến trúc được mô tả theo mô hình C4 [11] ở ba cấp: Context cho thấy ranh giới hệ thống và các tác nhân bên ngoài, Container cho thấy các thành phần chạy độc lập, Component cho thấy các module trong từng container. Cấp Code được bỏ vì chi tiết lớp và hàm đã nằm trong mã nguồn và đặc tả OpenAPI. Hình 3.1 tóm tắt toàn bộ kiến trúc trên một trang.

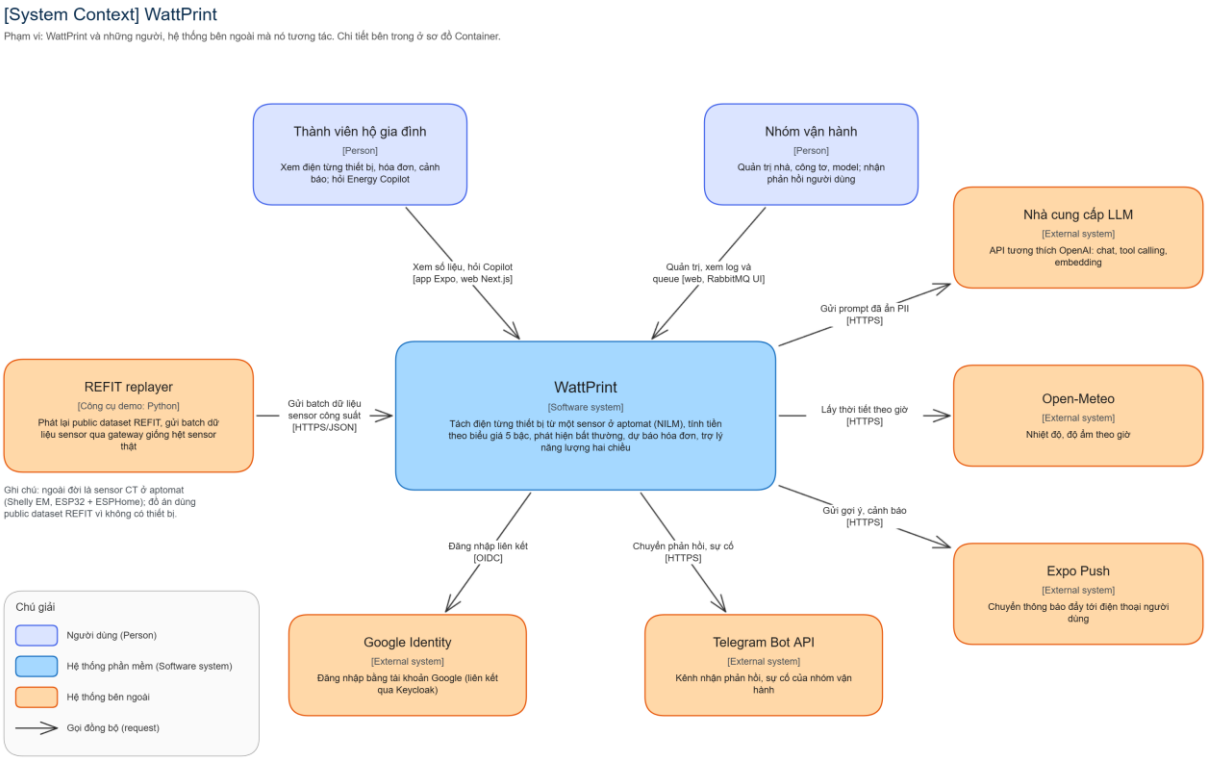
Ứng dụng client và REFIT replayer gọi vào một cổng duy nhất là API Gateway (Envoy). Gateway xác thực JWT rồi định tuyến tới bốn service nghiệp vụ. Các service gọi nhau đồng bộ theo một chiều `copilot → insight → core → nilm` và đẩy dữ liệu ngược lại bằng event qua RabbitMQ. Mỗi service có database riêng, tất cả nằm trên một PostgreSQL server dùng chung.



Hình 3.1. Kiến trúc tổng quan của WattPrint. Mũi tên liền là call đồng bộ, mũi tên đứt là sự kiện bất đồng bộ qua RabbitMQ

3.2. Bối cảnh hệ thống

Hình 3.2 coi WattPrint là một khối duy nhất giữa hai nhóm người dùng, nguồn dữ liệu sensor và các hệ thống ngoài ở mục 2.1. Sơ đồ nêu hai nguyên tắc tại ranh giới. Thứ nhất, WattPrint không tự lưu mật khẩu: đăng nhập đi qua Keycloak, liên kết Google Identity khi người dùng chọn đăng nhập bằng Google. Thứ hai, nhà cung cấp LLM nằm ngoài `trust boundary`, nên dữ liệu gửi đi phải được ẩn PII trước (BR9).



Hình 3.2. Sơ đồ System Context của WattPrint

3.3. Tách hệ thống thành các service

3.3.1. Tiêu chí tách service

Nhóm tách monolith theo ba tiêu chí, xếp theo thứ tự ưu tiên.

Tiêu chí đầu tiên là data ownership. Dữ liệu vận hành (hộ, dữ liệu sensor, hóa đơn), inference học máy, phân tích thống kê và tương tác AI nằm trên các nhóm bảng riêng, mảng này chỉ đọc kết quả của mảng kia chứ không sửa chéo. Đây là tiêu chí chính vì mỗi microservice cần tự chủ dữ liệu của mình [12].

Tiêu chí thứ hai là workload. Inference NILMFormer chạy PyTorch, tốn nhiều CPU và RAM. Ingest dữ liệu sensor cần phản hồi nhanh và ổn định. Phân tích chạy theo batch và lịch định kỳ. Copilot phần lớn thời gian chờ phản hồi từ LLM bên ngoài.

Tiêu chí thứ ba là fault isolation. LLM bên ngoài có độ trễ cao và hay lỗi mạng, nên phải tách để sự cố này không chặn ingest hay dashboard. Khi model NILM quá tải, ingest vẫn lưu dữ liệu mới và đánh dấu batch chờ xử lý.

3.3.2. Các phương án đã cân nhắc

Bảng 3.1 so sánh ba mức độ chia. Nhóm chọn phương án B.

Bảng 3.1. Các phương án tách service

Phương án	Cách chia	Nhận xét
A. Hai service	Gộp core với insight. Gộp nilm với copilot	Không đạt tối thiểu ba service nghiệp vụ của đề bài, và gộp hai khối có tải trái ngược: nilm tốn CPU/RAM, copilot chủ yếu chờ I/O mạng từ LLM
B. Bốn service (chọn)	core, nilm, insight, copilot	Mỗi service có miền dữ liệu, kiểu tải và đặc thù lỗi riêng. Số lượng vừa với nhóm ba người triển khai và vận hành
C. Bảy service trở lên	Tách thêm billing, device, alert, notification	Mỗi service chỉ có vài bảng nhưng làm tăng số database, call mạng và chi phí vận hành. Nghiệp vụ billing cần đọc trực tiếp chuỗi dữ liệu sensor của core nên phụ thuộc dữ liệu rất chặt, tách riêng chỉ thêm bước gọi mạng mà không tăng tính độc lập

3.3.3. Bốn service sau khi tách

Bảng 3.2 là kết quả: bốn service, mỗi service có database, trách nhiệm và chiều gọi đồng bộ riêng.

Bảng 3.2. Service, trách nhiệm, dữ liệu riêng và đối tượng gọi tới

Service	Trách nhiệm	Dữ liệu riêng	Gọi tới
core	Quản lý hộ gia đình, thành viên, mã mời, ghép công tơ, thiết bị, nhận batch dữ liệu sensor, biểu giá 5 bậc và tính hóa đơn chính thức. Là `source of truth`	core_db (TimescaleDB)	nilm
nilm	Chạy inference NILMFormer, hiệu chỉnh tham số theo hộ, quản lý phiên bản model	nilm_db và volume chứa model	Không gọi service nào
insight	Duy trì read model tiêu thụ, phát hiện bất thường và tiêu thụ ngầm, dự báo hóa đơn và bậc giá, ngân sách, so sánh hộ tương đồng, what-if, M&V, API truy vấn SQL an toàn	insight_db (TimescaleDB)	core, Open-Meteo
copilot	Hỏi đáp tự nhiên, gợi ý chủ động, guardrail ẩn PII và chống prompt injection, verifier, RAG, xếp hạng theo phản hồi, thông báo, thống kê token, điều phối kế hoạch tiết kiệm	copilot_db (pgvector)	insight, LLM, Expo Push, Telegram

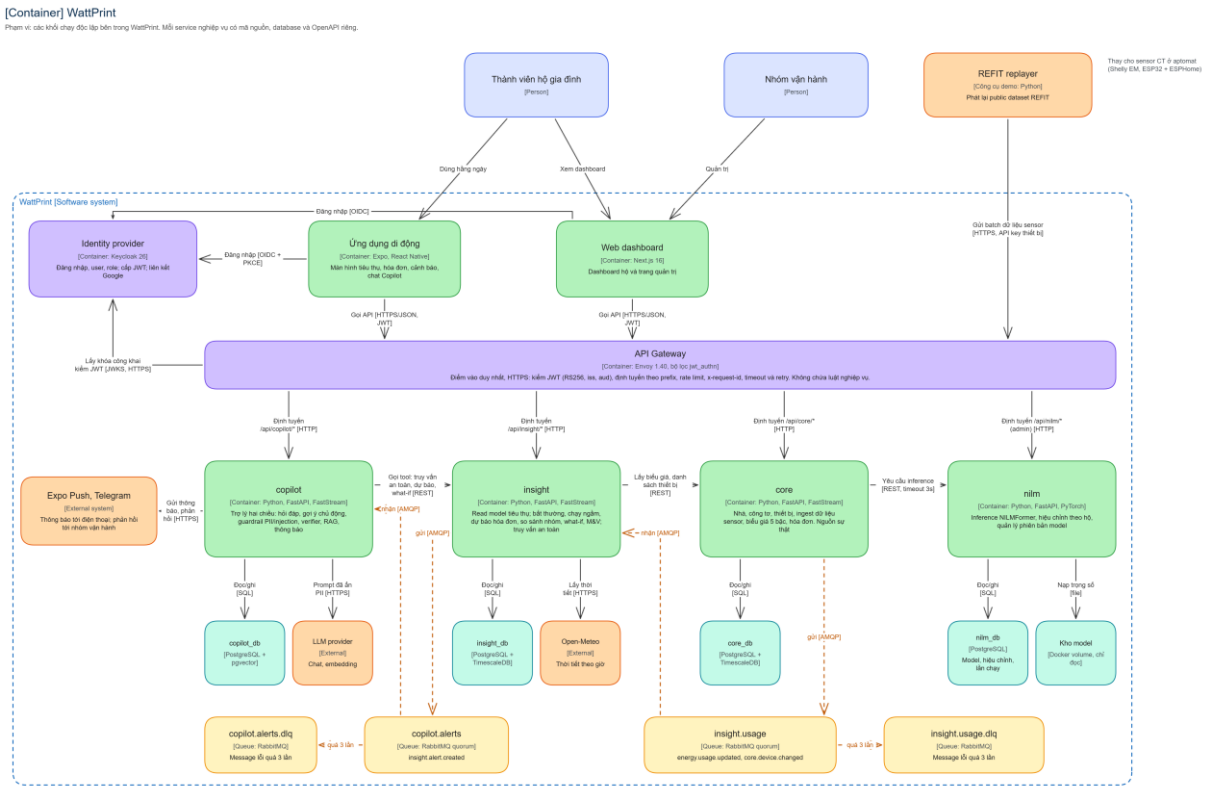
Bảng 3.2 kéo theo ba quyết định cần nói rõ. Thứ nhất, hóa đơn chính thức nằm ở `core` vì phải tính trực tiếp từ dữ liệu sensor gốc và phiên bản biểu giá để nhất quán với `source of truth`, còn dự báo hóa đơn chỉ là ước tính nên nằm ở `insight`. Thứ hai, `nilm` không gọi service nào khác, chỉ nhận công suất tổng và trả kết quả tách tải, nên là điểm cuối của chuỗi gọi. Thứ ba, `copilot` không đọc dữ liệu tiêu thụ từ `core` mà đi qua `insight` (lý do ở mục 3.7).

Call đồng bộ chỉ đi một chiều `copilot → insight → core → nilm`. Nhờ vậy không có `circular dependency`, và một service ngừng chỉ ảnh hưởng các call đến từ tầng ngay phía trên nó.

3.4. Các khối triển khai

Hình 3.3 là sơ đồ Container. Ngoài bốn service do nhóm viết, hệ thống dùng bốn phần mềm có sẵn: Envoy làm gateway, Keycloak quản lý định danh, RabbitMQ làm message broker và PostgreSQL làm database. Tổng cộng tám container, chưa kể hai ứng dụng client chạy ngoài Compose. Mỗi service nghiệp vụ là một container duy nhất: HTTP router, consumer, outbox relay và các job định kỳ chạy chung một process dưới dạng background task, để số container vừa đủ cho một máy mà không phải tách worker riêng. Docker Compose là môi trường demo của bài tập. Mỗi service đã có Dockerfile, nhận cấu hình và địa chỉ service phía sau qua biến môi trường, và có `/health`, `/ready` làm probe, nên đưa từng service lên orchestrator như Kubernetes và deploy độc lập không cần đổi mã nguồn.

Theo quy ước C4, mỗi queue là một container lưu message, còn RabbitMQ là chi tiết hạ tầng bên dưới [11]. Vì vậy Hình 3.3 có bốn khối queue màu vàng: `insight.usage`, `copilot.alerts` và hai DLQ `.dlq` đi kèm.



Hình 3.3. Sơ đồ Container của WattPrint. Nét đứt là message AMQP: mũi tên đi tới queue là bên gửi, đi ra khỏi queue là bên nhận

3.5. Giao tiếp giữa các service

3.5.1. Gọi đồng bộ qua REST

Call cần kết quả ngay (`core` gọi `nilm` chạy inference, `insight` gọi `core` lấy biểu giá và thiết bị, `copilot` gọi `insight` lấy số liệu phân tích) dùng REST qua HTTP, đi thẳng trong mạng nội bộ của Compose, không qua gateway. Service gọi chuyển tiếp `X-User-Id`, `X-User-Roles` và `X-Request-ID`, còn địa chỉ service đích lấy từ biến môi trường. Phía gọi luôn đặt timeout, retry có giới hạn và phân biệt lỗi nghiệp vụ (4xx, giữ nguyên) với lỗi hạ tầng (timeout trả 504, mất kết nối hoặc circuit breaker mở trả 503). Thông số fault tolerance chi tiết thuộc chương về độ tin cậy.

3.5.2. Sự kiện bất đồng bộ qua RabbitMQ

Có những thông tin bên gửi không cần chờ. Khi `core` lưu xong kết quả tách tải, `core` không cần đợi `insight` cập nhật read model. Khi `insight` phát hiện bất thường, `insight` không cần chờ `copilot` gọi LLM soạn gợi ý xong. Hai luồng này dùng event qua RabbitMQ (Bảng 3.3). Mỗi consumer nghe trên một quorum queue [13] với `delivery-limit = 3` và một DLQ đi kèm.

Bảng 3.3. Các sự kiện trên exchange `wattprint.events` (kiểu topic)

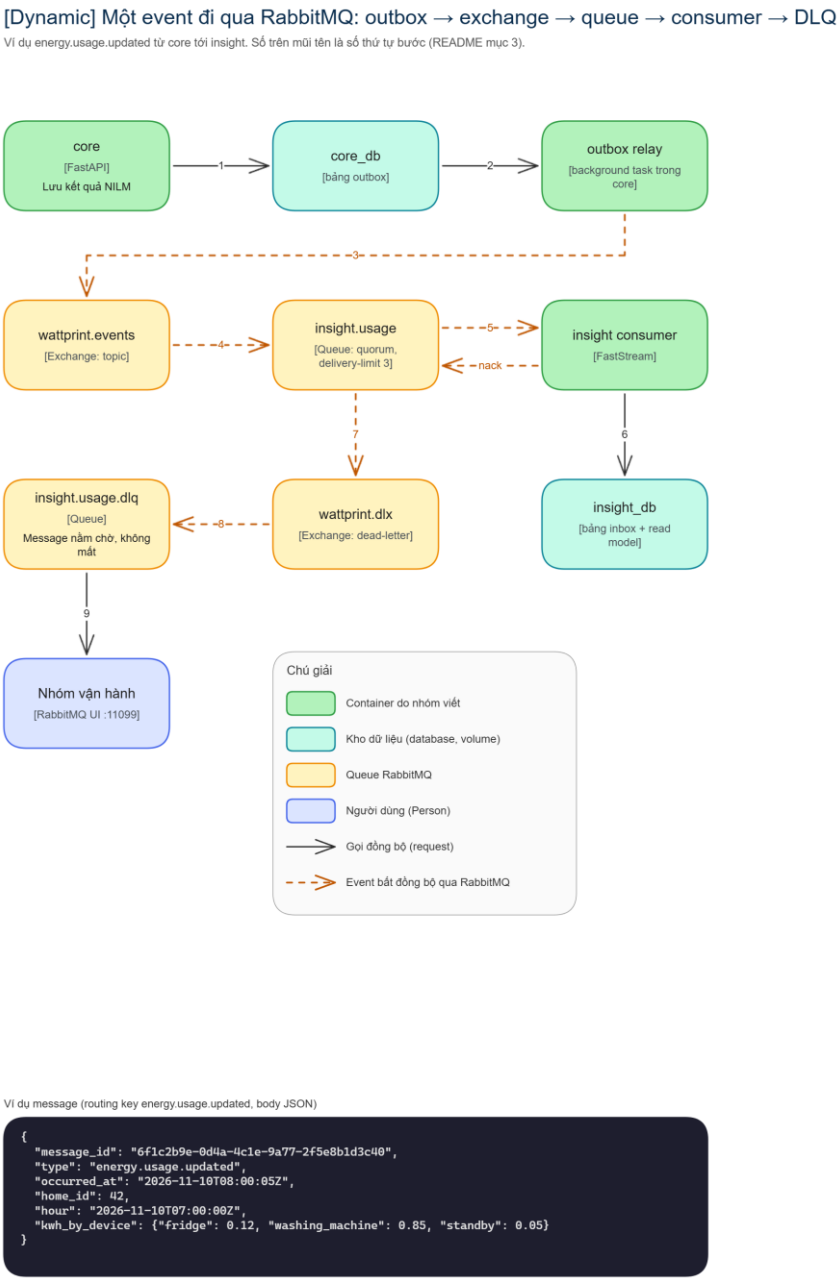
Routing key	Bên phát	Bên nhận	Mục đích
`energy.usage.updated`	core	insight	Thông báo có kết quả ước tính mới, kèm tổng điện năng theo giờ của từng thiết bị
`core.device.changed`	core	insight	Đồng bộ thay đổi danh mục thiết bị sang read model của insight
`insight.alert.created`	insight	copilot	Thông báo có cảnh báo mới để kích hoạt luồng gợi ý chủ động

Một event đi qua chín bước. Hình 3.4 minh họa với `energy.usage.updated` từ `core` tới `insight`:

1. `core` lưu kết quả NILM và một dòng vào bảng `outbox` trong cùng một transaction.
2. `OutboxRelay` (background task trong `core`) đọc các dòng chưa gửi bằng `SELECT ... FOR UPDATE SKIP LOCKED`, nên chạy nhiều instance cũng không gửi trùng.
3. Relay publish lên exchange `wattprint.events` kèm routing key, đợi broker confirm rồi mới đánh dấu đã gửi.
4. Exchange chép message vào mọi queue có binding khớp. `insight.usage` bind `energy.usage.*` và `core.device.*`.
5. RabbitMQ giao message cho consumer của `insight`. Xử lý lỗi thì consumer `nack` để broker redeliver.

6. Consumer tra `message_id` trong bảng `inbox`. Đã có thì bỏ qua, chưa có thì cập nhật read model và ghi `inbox` trong cùng một transaction rồi `ack`.
7. Message bị redeliver quá ba lần vẫn lỗi được quorum queue chuyển sang dead-letter exchange `wattprint.dlx`.
8. `wattprint.dlx` đẩy message vào `insight.usage.dlq`, message nằm đó chứ không mất.
9. Nhóm vận hành xem DLQ trên giao diện quản trị RabbitMQ, sửa nguyên nhân rồi đẩy message về `insight.usage` để chạy lại.

Outbox và inbox xử lý hai rủi ro khác nhau. Nếu lưu database rồi publish riêng, process sập giữa hai bước sẽ làm `insight` thiếu dữ liệu. Ghi event vào `outbox` cùng transaction với dữ liệu nghiệp vụ loại bỏ rủi ro này, theo pattern Transactional Outbox [14]. Phía nhận, RabbitMQ giao at-least-once nên một message có thể đến hai lần. Bảng `inbox` lưu `message_id` đã xử lý để consumer bỏ qua bản trùng, nhờ đó consumer idempotent.



Hình 3.4. Một sự kiện đi qua RabbitMQ: outbox, exchange, queue, consumer, DLQ

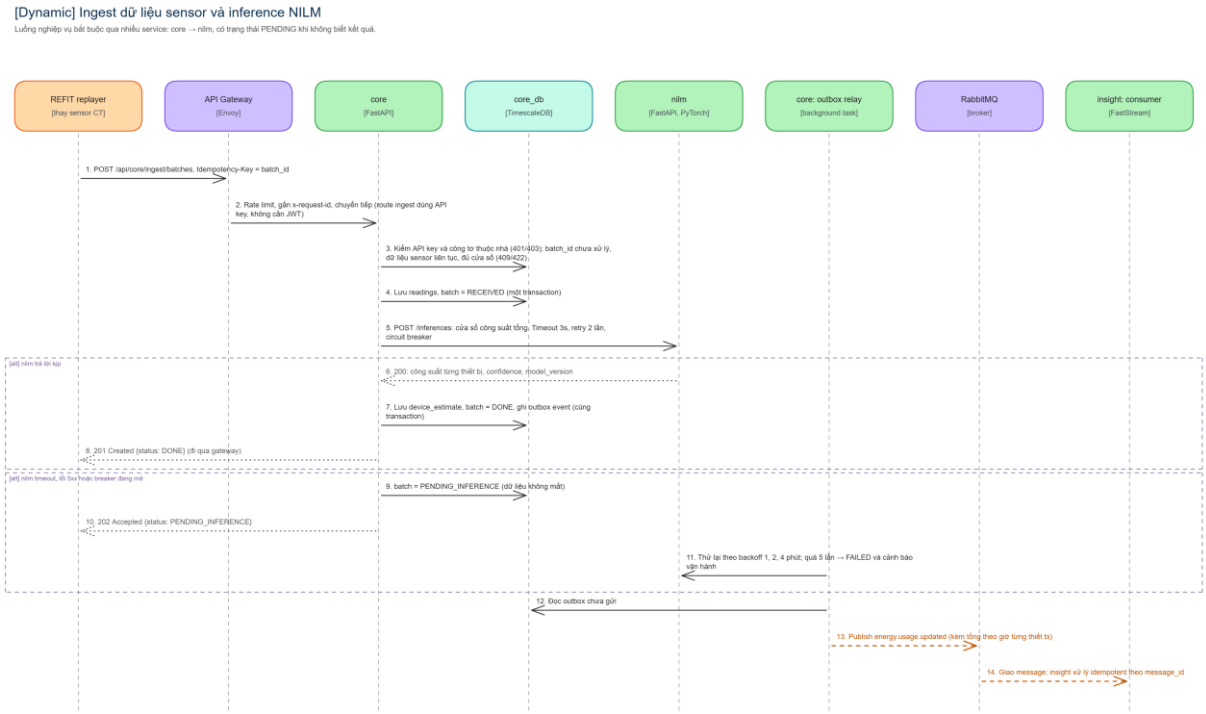
3.6. Luồng nghiệp vụ nhiều bước và xử lý lỗi giữa chừng

Đề bài yêu cầu ít nhất một luồng đi qua hai service và có cách xử lý khi lỗi giữa chừng. WattPrint có hai luồng như vậy. Mỗi luồng xử lý khác nhau tùy bước đầu có rollback được hay không.

3.6.1. Ingest dữ liệu sensor và chạy inference NILM

Khi nhận một batch, `core` kiểm tra API key thiết bị, công tơ thuộc hộ và tính liên tục của batch (BR2, BR3), lưu dữ liệu sensor thô rồi gọi đồng bộ sang `nilm`. Nếu `nilm` trả lời kịp, `core` lưu công suất ước tính từng thiết bị, ghi event vào `outbox` và trả 201 với trạng thái `DONE`. Nếu call `nilm` bị timeout, trả 5xx hoặc gặp lúc circuit breaker mở, `core`

chưa có kết quả inference. `core` không rollback dữ liệu sensor đã lưu vì đây là dữ liệu thật, không được mất. Thay vào đó, `core` đánh dấu batch `PENDING_INFERENCE`, trả 202 và để một background task retry theo backoff 1, 2, 4 phút. Quá năm lần retry vẫn lỗi thì batch chuyển sang `FAILED` và `core` cảnh báo nhóm vận hành (Hình 3.5).

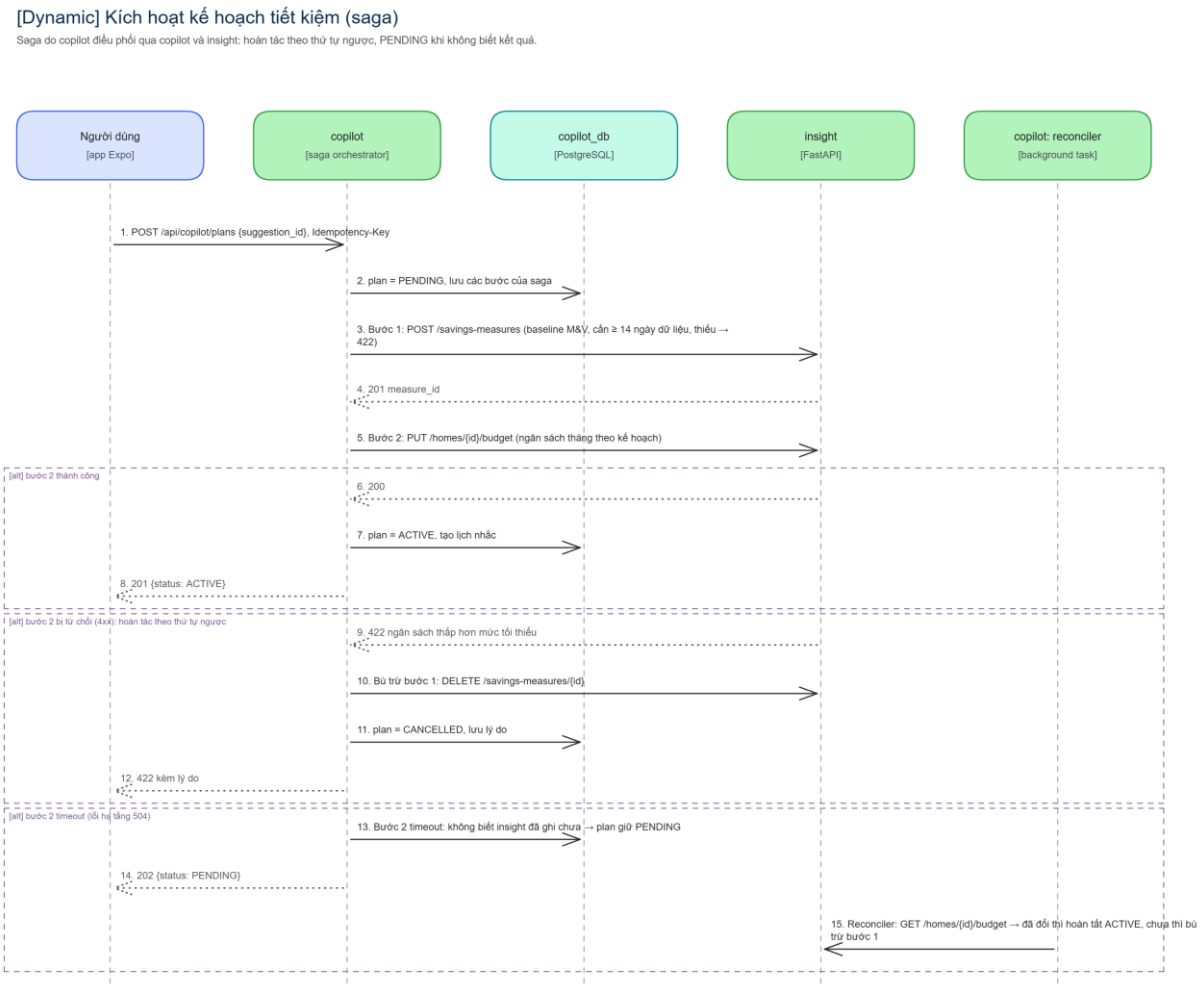


Hình 3.5. Luồng ingest dữ liệu sensor và chạy inference NILM, có trạng thái PENDING_INFERENCE khi nilm không phản hồi

3.6.2. Kích hoạt kế hoạch tiết kiệm

Khi người dùng chấp nhận một gợi ý tiết kiệm, `copilot` điều phối một saga gồm hai bước qua `insight`. Bước một tạo baseline M&V ở `insight` (cần tối thiểu 14 ngày dữ liệu theo BR5, thiếu thì `insight` trả 422). Bước hai đặt ngân sách tháng theo kế hoạch. Saga rẽ ba nhánh tùy kết quả bước hai (Hình 3.6). Thành công thì kế hoạch chuyển `ACTIVE`. Nếu `insight` từ chối bằng lỗi nghiệp vụ 4xx, `copilot` compensate theo thứ tự ngược bằng cách gọi API xóa baseline ở bước một, rồi đánh dấu kế hoạch `CANCELLED` kèm lý do. Nếu bước hai timeout (lỗi hạ tầng, 504), `copilot` không biết `insight` đã lưu ngân sách hay chưa nên giữ kế hoạch ở `PENDING`, và background job `reconciler` hỏi lại `insight` để hoàn tất hoặc compensate sau.

Saga dựa trên việc phân biệt hai loại lỗi. Lỗi nghiệp vụ 4xx cho biết chắc bước hai chưa đổi dữ liệu nên compensate ngay. Lỗi hạ tầng thì không biết gì, nên phải giữ trạng thái chờ rồi reconcile.



Hình 3.6. Luồng kích hoạt kế hoạch tiết kiệm (saga): thành công, hoàn tác theo thứ tự ngược và PENDING

3.7. Dữ liệu

Dữ liệu nằm trên một PostgreSQL server với năm database: `core_db`, `insight_db`, `copilot_db`, `nilm_db` và `keycloak_db`. Mỗi database có một role riêng, không service nào có quyền vào database của service khác. Cách này giữ tám container mà vẫn tách dữ liệu theo pattern Database per Service [12]. Khi cần, mỗi database chuyển sang server riêng bằng cách đổi chuỗi kết nối trong biến môi trường.

Server chạy PostgreSQL 18 với hai extension. TimescaleDB xử lý chuỗi thời gian ở `core_db` (lưu dữ liệu sensor, tổng hợp theo giờ và ngày, nén dữ liệu cũ, xóa chunk hết hạn) và ở `insight_db` (read model tiêu thụ). `pgvector` phục vụ tìm kiếm vector cho RAG ở `copilot_db`.

Vì không service nào đọc database của service khác, nhu cầu đọc chéo được giải quyết bằng một trong hai cách:

- Dùng event theo pattern CQRS [14]. `insight` giữ một read model dữ liệu tiêu thụ đã tổng hợp theo giờ, cập nhật bất đồng bộ từ event của `core`. `core` lo luồng ghi và là `source of truth`, `insight` tổ chức dữ liệu cho truy vấn phân tích.
- Gọi REST API khi cần dữ liệu mới nhất hoặc dữ liệu cấu hình nhỏ. Service cần dữ liệu hỏi thẳng service sở hữu, ví dụ `insight` lấy biểu giá và danh mục thiết bị từ `core`.

Cách tách này quyết định độ an toàn của Text-to-SQL ở Copilot. Để trả lời câu hỏi tự do về điện năng, LLM trong `copilot` tự sinh câu SQL. Nếu câu SQL chạy thẳng trên database vận hành của `core`, một truy vấn sai hoặc bị prompt injection có thể chạm vào dữ liệu nguồn. Vì vậy `copilot` gửi câu SQL tới endpoint truy vấn an toàn của `insight`. Tại đây `SQL guard` kiểm tra cú pháp để chỉ cho `SELECT` trên các view được cấp quyền, tự ép điều kiện `home_id` của người hỏi, đặt timeout 2 giây và chỉ chạy trên `insight_db`. Các bảng của monolith cũng chuyển về đúng database theo Bảng 3.2: bảng phân tích và M&V về `insight_db`, bảng lịch sử inference (`inference_run`) về `nilm_db`, bảng hội thoại và tri thức RAG về `copilot_db`.

3.8. Gateway và bảo mật

Gateway là Envoy, cấu hình trong `envoy.yaml`. Theo đề bài, gateway chỉ định tuyến và áp chính sách ở biên, không chứa logic nghiệp vụ. Mọi chính sách đều là filter dựng sẵn của Envoy:

- Định tuyến theo tiền tố `/api/core/`, `/api/insight/`, `/api/copilot/` và `/api/nilm/` (chỉ admin), mỗi route có timeout và retry riêng. Envoy chỉ retry request `GET` khi lỗi kết nối, không retry `POST` để tránh ghi trùng.
- Filter `jwt_authn` kiểm chữ ký RS256, `issuer` và `audience` của JWT bằng tập khóa công khai (JWKS) lấy từ Keycloak [15]. Sau khi kiểm, Envoy gán `sub` và role vào `X-User-Id` và `X-User-Roles`, đồng thời xóa header trùng tên do client gửi, nên service phía sau tin được hai header này.
- Rate limit, giới hạn kích thước request body và gán `x-request-id` để theo dõi log xuyên suốt.
- Route nhận dữ liệu từ sensor không yêu cầu JWT vì thiết bị không đăng nhập. `core` tự xác thực API key thiết bị và trạng thái ghép nối công tơ (BR2).

Phân quyền theo hộ, theo gói Pro và theo role admin do từng service kiểm vì đó là quy tắc nghiệp vụ. Các service không mở cổng ra máy host, nên chỉ gateway và các service trong mạng nội bộ gọi tới được.

3.9. Công nghệ sử dụng

Nhóm chỉ có ba người nên nguyên tắc chọn công nghệ là giữ số công cụ nhỏ và ưu tiên cấu hình có sẵn hơn tự viết code (Bảng 3.4). Backend dùng Python, hai ứng dụng client dùng TypeScript.

Bảng 3.4. Công nghệ sử dụng và lý do chọn

Lớp	Chọn	Lý do
Gateway	Envoy 1.40	Kiểm JWT (RS256, JWKS), rate limit, request ID, timeout và retry là filter có sẵn trong bản mã nguồn mở, khai báo trong một file cấu hình, không phải viết code. Cùng proxy với Istio và Envoy Gateway nên khi lên Kubernetes không phải đổi gateway
Identity	Keycloak 26	Cấp JWT theo chuẩn OIDC, liên kết đăng nhập Google. Dùng chung `keycloak_db` trên PostgreSQL server thay vì cần container database riêng
Database	PostgreSQL 18, TimescaleDB 2.31, pgvector	Một server duy nhất đáp ứng cả dữ liệu quan hệ, chuỗi thời gian và tìm kiếm vector, không cần vận hành ba hệ cơ sở dữ liệu khác nhau
Message broker	RabbitMQ 4, quorum queue	Hỗ trợ sẵn giới hạn redelivery (`delivery-limit`) và dead-letter exchange, không cần tự viết cơ chế đếm retry
Service	Python 3.12, FastAPI, SQLAlchemy async, Alembic	Kế thừa trực tiếp mã nguồn sẵn có. FastAPI hỗ trợ bất đồng bộ và tự động sinh đặc tả OpenAPI
Messaging trong service	FastStream	Viết consumer tương tự như route của FastAPI, tự sinh tài liệu AsyncAPI và có sẵn broker trong bộ nhớ để chạy test
Gọi HTTP và fault tolerance	httpx, tenacity. Circuit breaker và bulkhead tự viết trong thư viện chung	Thư viện circuit breaker phổ biến chỉ hỗ trợ async qua Tornado, nên nhóm tự viết khoảng 30 dòng code và dùng `asyncio.Semaphore` làm bulkhead [16]
Mã nguồn	Monorepo uv workspace: `services/` và `libs/common`	Quản lý phụ thuộc bằng một lockfile. Phần fault tolerance, log, outbox và inbox chỉ cần viết một lần cho cả bốn service
Kiểm tra	ruff, pytest, Postman với Newman, GitHub Actions	Tự động chạy test, build image, bật cụm compose và chạy Postman collection trên pipeline CI

Nhóm không dùng Prometheus, Grafana, Loki, Jaeger, không tách container worker riêng và không dùng container database riêng cho Keycloak. Mỗi container ghi log JSON ra `stdout` kèm `request_id`, tra bằng `docker compose logs` là đủ để truy vết một request qua các service.

3.10. Thành phần bên trong từng service

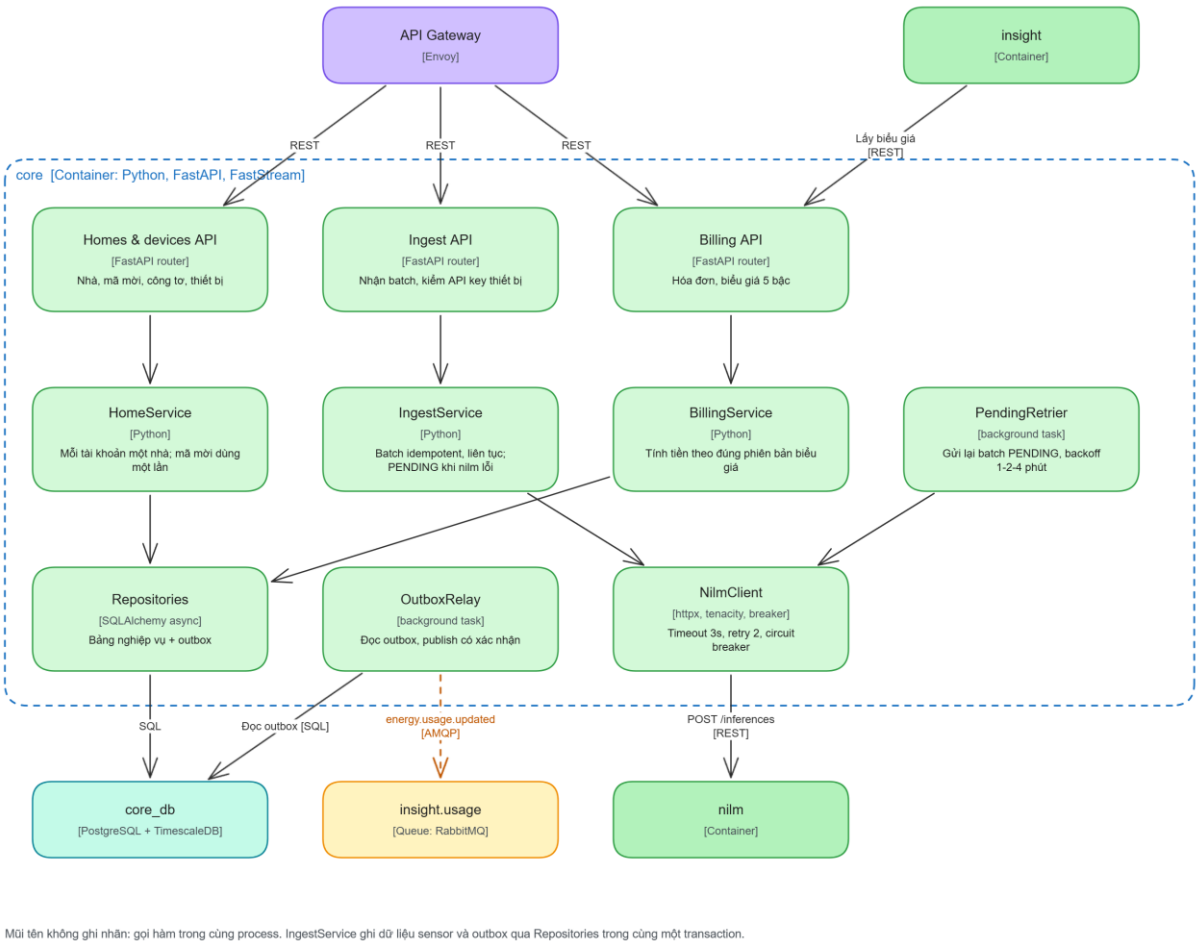
Các sơ đồ Component cho thấy module bên trong từng container và các điểm giao tiếp ra ngoài. Mũi tên không nhãn là call hàm trong cùng process. Màu sắc theo quy ước chung: xanh lá là phần nhóm viết, tím là phần mềm có sẵn, vàng là queue, xanh ngọc là kho dữ liệu, cam là hệ thống ngoài, nét đứt là event bất đồng bộ.

core (Hình 3.7) có ba cụm API: quản lý hộ và thiết bị, ingest dữ liệu sensor và hóa đơn. `IngestService` ghi dữ liệu sensor và event qua `Repositories` trong cùng một transaction. `OutboxRelay` và `PendingRetrier` là hai background task: một đẩy event lên RabbitMQ, một retry các batch đang chờ inference. `NilmClient` là chỗ duy nhất `core` gọi service khác, nên timeout, retry và circuit breaker tập trung ở đó.

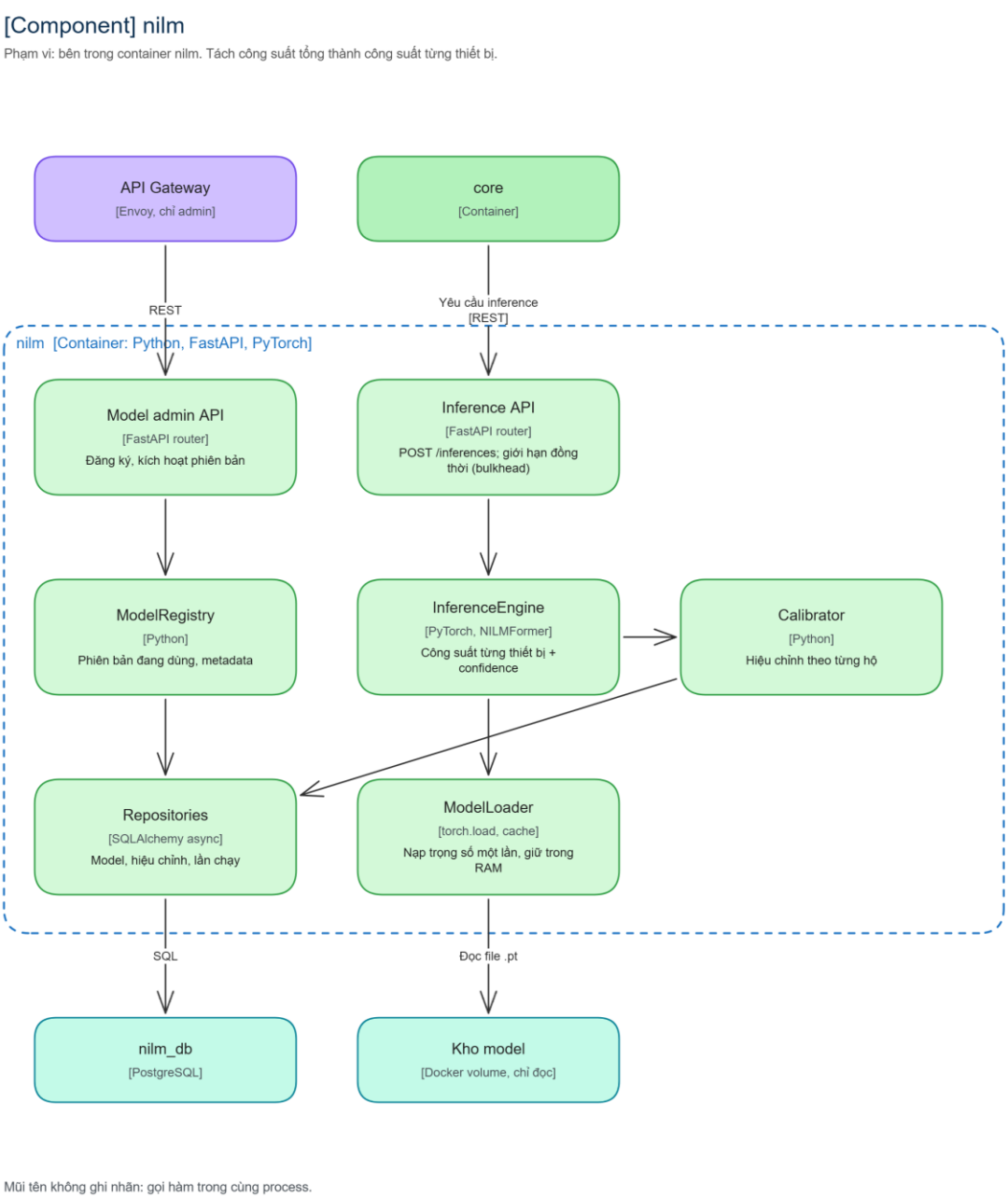
nilm (Hình 3.8) tách thành API inference và API quản lý phiên bản model (chỉ admin). `InferenceEngine` chạy NILMFormer, `Calibrator` hiệu chỉnh theo từng hộ, `ModelLoader` nạp trọng số một lần rồi giữ trong RAM. Endpoint inference dùng bulkhead để giới hạn số request chạy đồng thời, tránh một đợt tải cao làm cạn bộ nhớ.

insight (Hình 3.9) nhận event qua `Usage consumer` để cập nhật read model. Các `Detectors` kiểm tra phụ tải bất thường, chạm bậc giá và vượt ngân sách, rồi phát `insight.alert.created` qua `OutboxRelay`. Phía API có các endpoint phân tích, dự báo và `SQL guard` bảo vệ endpoint truy vấn an toàn (mục 3.7).

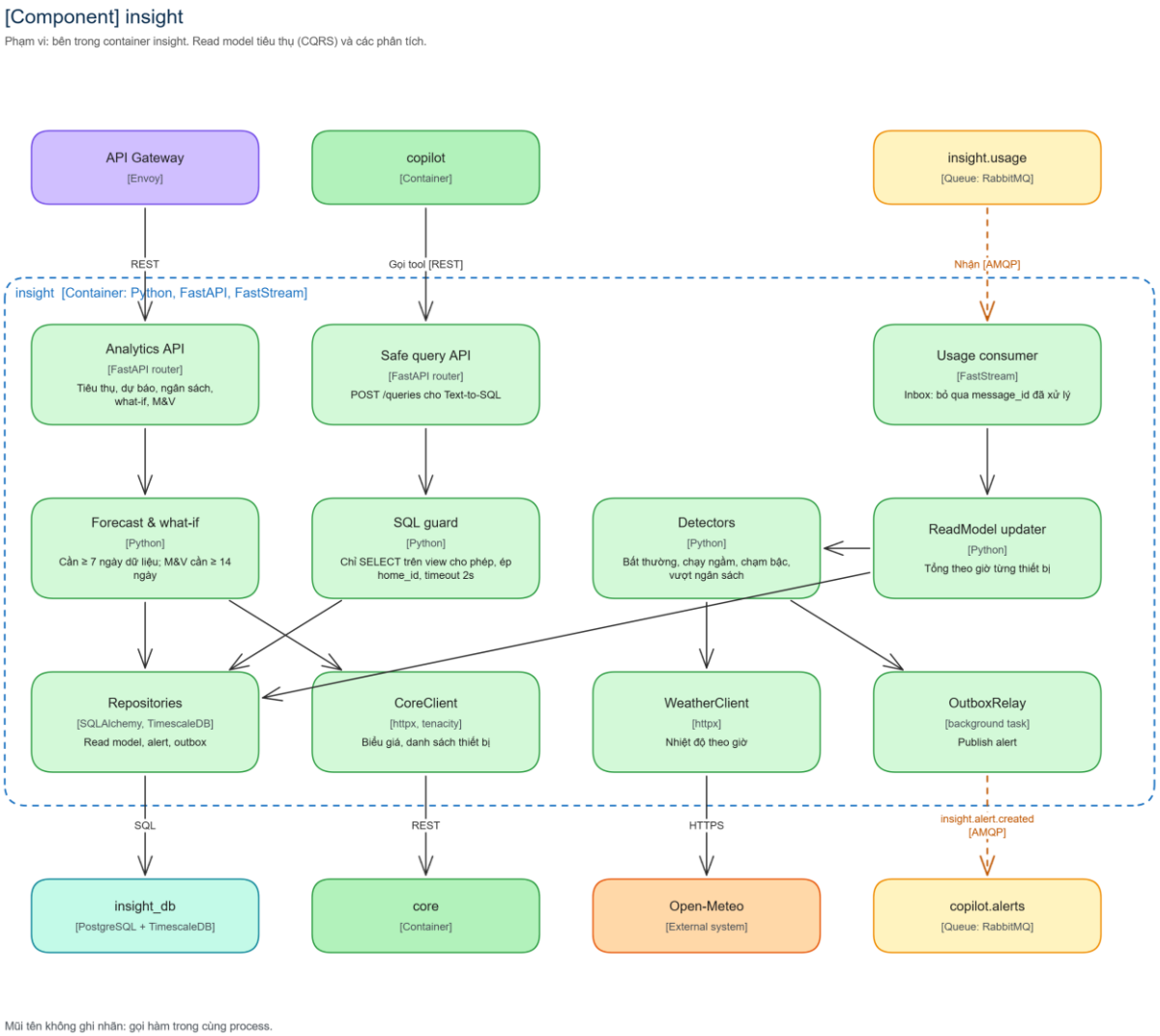
copilot (Hình 3.10) có hai đường vào. Đường hỏi đáp đi từ `Chat API` qua `Guardrail` (ẩn PII, chặn prompt injection, kiểm tra gói Pro) tới `Orchestrator`, nơi LLM gọi tool lấy số liệu từ `insight`, rồi `Verifier` kiểm tra số liệu, đơn vị và nguồn trích dẫn trước khi trả lời. Đường gợi ý chủ động đi từ `Alert consumer` qua `Ranker` (xếp hạng theo phản hồi, lọc khung giờ nghỉ, chống gửi trùng) tới `Notifier`. `Saga orchestrator` điều phối luồng kích hoạt kế hoạch tiết kiệm (mục 3.6.2).



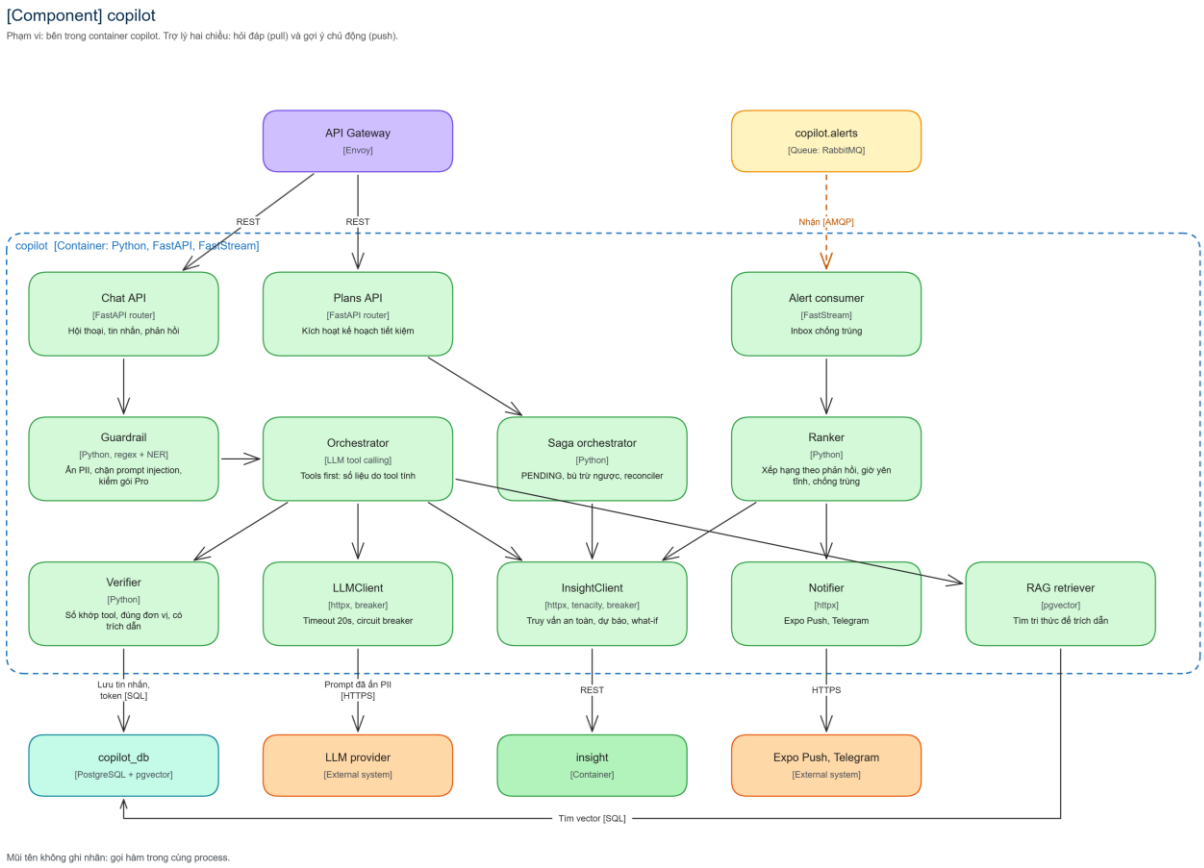
Hình 3.7. Sơ đồ Component của service core



Hình 3.8. Sơ đồ Component của service nilm



Hình 3.9. Sơ đồ Component của service insight



Hình 3.10. Sơ đồ Component của service copilot

CHƯƠNG 4. ĐÁNH ĐỔI VÀ RỦI RO CỦA KIẾN TRÚC

4.1. Các quyết định chính và đánh đổi

Mỗi quyết định ở Chương 3 đều có cái giá. Bảng 4.1 ghi các quyết định lớn, lý do chọn và đánh đổi nhóm chấp nhận.

Bảng 4.1. Quyết định kiến trúc và đánh đổi

Quyết định	Lý do	Đánh đổi chấp nhận
Tách bốn service thay vì giữ kiến trúc monolith	Mỗi service có ranh giới dữ liệu, đặc thù tải và kiểu lỗi riêng. Đáp ứng yêu cầu đề bài	Phát sinh độ trễ gọi qua mạng, thêm điểm có nguy cơ lỗi cục bộ và gỡ lỗi phức tạp hơn một process
Dừng ở bốn service thay vì tách thành bảy service	Giữ số lượng container, database và call mạng ở mức nhóm ba người kiểm soát tốt	Phạm vi mỗi service lớn hơn, chẳng hạn core đảm nhận cả quản lý hộ gia đình lẫn tính hóa đơn
Quy định chiều gọi đồng bộ một chiều	Không có circular dependency. Sự cố ở một service chỉ ảnh hưởng tầng ngay phía trên	Copilot phải đi vòng qua insight để lấy dữ liệu tiêu thụ, thêm một chặng gọi mạng so với đọc trực tiếp
Truyền sự kiện qua RabbitMQ cho hai luồng phụ	Bên gửi không phải chờ bên nhận xử lý xong. Có sẵn DLQ để lưu giữ message lỗi và chạy lại	Read model của insight có độ trễ ngắn so với core (eventual consistency). Phải duy trì thêm bảng outbox và inbox
Dùng Envoy làm gateway thay vì Nginx	Kiểm JWT từ Keycloak là filter có sẵn. Module `auth_jwt` của Nginx chỉ có trong gói thương mại [17], bản mã nguồn mở phải ghép thêm oauth2-proxy hoặc module ngoài	File `envoy.yaml` dài và ít quen thuộc hơn cấu hình Nginx. Cấu hình tĩnh nên đổi route hoặc chính sách phải restart container gateway
Dùng một PostgreSQL server cho năm database	Giữ tổng số container ở mức tám trong khi vẫn tách biệt quyền theo từng database role	PostgreSQL server là điểm lỗi chung (single point of failure) về hạ tầng, các service chia sẻ tài nguyên trên cùng server
Bốn service cùng ngôn ngữ Python và chung thư viện	Viết phần fault tolerance, log, outbox và inbox một lần. Giảm số công nghệ phải học	Backend gắn chặt với hệ sinh thái Python. Cập nhật thư viện chung ảnh hưởng tới cả bốn service
Mỗi service chạy trong một container (API, consumer, relay chung process)	Dễ khởi chạy và gom toàn bộ log của một service về một luồng duy nhất	Không scale riêng được phần API và phần background job. Các job nền nặng cạnh tranh tài nguyên với API
Tự viết circuit breaker và bulkhead trong thư viện chung	Thư viện sẵn có không tương thích gọn với mô hình async của FastAPI	Phải tự viết bộ kiểm thử kỹ lưỡng và thiếu các tính năng giám sát nâng cao của những thư viện phổ biến

4.2. Rủi ro còn lại và hướng xử lý

Bảng 4.2 liệt kê các rủi ro đã nhận diện ở mức kiến trúc và cách xử lý. Cấu hình chi tiết và số liệu thực nghiệm sẽ nằm ở các chương về độ tin cậy và đánh giá.

Bảng 4.2. Rủi ro và hướng xử lý

Rủi ro	Biểu hiện	Hướng xử lý
Service nilm quá tải hoặc ngừng phản hồi	core không nhận được kết quả chạy inference ngay khi lưu batch	Giữ nguyên dữ liệu sensor gốc, đánh dấu batch `PENDING_INFERENCE` và dùng background job retry có backoff. Kết hợp circuit breaker ở core và bulkhead ở nilm
Nhà cung cấp LLM phản hồi chậm hoặc lỗi	Copilot không trả lời kịp. Không được kéo theo nghẽn dashboard	Áp timeout 20 giây, mở circuit breaker và trả mã 503. Dashboard và phân tích không phụ thuộc vào LLM
Message đến lặp lại hoặc xử lý thất bại	Read model của insight bị cộng trùng hoặc thiếu dữ liệu mới	Dùng bảng inbox chống xử lý trùng, bảng outbox chống mất sự kiện và đẩy message lỗi vào DLQ để kiểm tra, chạy lại
Read model của insight lệch so với core	Số liệu trên biểu đồ phân tích khác với số liệu hóa đơn	Quy định số liệu hóa đơn chính thức luôn lấy trực tiếp từ core. Cơ chế đồng bộ lại read model khi lệch sẽ được xác định ở phần triển khai
Copilot trả lời sai số liệu thực tế	Người dùng nhận phải con số tiền điện hoặc công suất sai	Mọi con số đều do tool tính toán từ dữ liệu hộ gia đình. Module Verifier loại bỏ câu trả lời sai số liệu, đơn vị hoặc nguồn (BR8)
Lộ thông tin cá nhân qua LLM hoặc Text-to-SQL	Prompt gửi đi chứa PII hoặc câu SQL tự sinh chạm dữ liệu nguồn	Module Guardrail ẩn PII trước khi gọi LLM. Truy vấn SQL chạy qua endpoint của insight với SQL guard trên database phân tích riêng
Container PostgreSQL ngừng hoạt động	Mọi service mất kết nối dữ liệu cùng lúc	Chấp nhận trong phạm vi bài tập lớn. Hướng nâng cấp thực tế là tách database sang server riêng hoặc cấu hình replica dự phòng

TÀI LIỆU THAM KHẢO

[1] C. Ta, "Usage information to save electricity in Vietnam," International Growth Centre (IGC), 2025. https://www.theigc.org/blogs/usage-information-vietnam-save-electricity

[2] S. Z. Attari, M. L. DeKay, C. I. Davidson, W. Bruine de Bruin, "Public perceptions of energy consumption and savings," PNAS, vol. 107, no. 37, 2010. https://pmc.ncbi.nlm.nih.gov/articles/PMC2941272/

[3] Báo Sài Gòn Giải Phóng, "Tra cứu tiền điện hằng ngày trên app dùng chung của EVN," 22/06/2026. https://www.sggp.org.vn/tra-cuu-tien-dien-hang-ngay-tren-app-dung-chung-cua-evn-post858550.html

[4] Tuya.vn, "Ổ cắm WiFi đo điện TC1," giá niêm yết 235.000 đồng. https://tuya.vn/o-cam-c-smart-series-wifi-cong-suat-lon-16a-tuya-tc1-p40701967.html

[5] Home Assistant, "Home Assistant Green," giá niêm yết 199 USD. https://www.home-assistant.io/green/

[6] G. W. Hart, "Nonintrusive appliance load monitoring," Proceedings of the IEEE, vol. 80, no. 12, pp. 1870-1891, 1992.

[7] A. Petralia, P. Charpentier, Y. Kadhi, T. Palpanas, "NILMFormer: Non-Intrusive Load Monitoring that Accounts for Non-Stationarity," ACM SIGKDD, 2025. arXiv:2506.05880.

[8] P. Buckley, "Prices, information and nudges for residential electricity conservation: A meta-analysis," Ecological Economics, vol. 172, 2020.

[9] N. N. Nguyen, D. Goto, D. Tran, H. T. Vu, "Nudging households to save electricity with feedback: Experimental evidence from Vietnam," Energy Policy, vol. 195, 2024.

[10] D. Murray, L. Stankovic, V. Stankovic, "An electrical load measurements dataset of United Kingdom households from a two-year longitudinal study," Scientific Data, vol. 4, 160122, 2017.

[11] S. Brown, "The C4 model for visualising software architecture." https://c4model.com

[12] S. Newman, Building Microservices, 2nd ed., O'Reilly Media, 2021.

[13] Broadcom, "Quorum Queues," tài liệu RabbitMQ. https://www.rabbitmq.com/docs/quorum-queues

[14] C. Richardson, Microservices Patterns, Manning Publications, 2018.

[15] Envoy Project, "JWT Authentication" (http_filters/jwt_authn_filter), tài liệu Envoy. https://www.envoyproxy.io/docs/envoy/latest/configuration/http/http_filters/jwt_authn_filter

[16] M. T. Nygard, Release It!, 2nd ed., Pragmatic Bookshelf, 2018.

[17] F5, "Module ngx_http_auth_jwt_module," tài liệu Nginx. https://nginx.org/en/docs/http/ngx_http_auth_jwt_module.html